



CAPS: Compositional Attack Path Scoring for LLM Deployment Stacks

1

Abstract: Evaluating the security posture of Large Language Model (LLM) deployment stacks is a critical challenge in modern artificial intelligence security. Traditional vulnerability management frameworks—such as the Common Vulnerability Scoring System (CVSS) and component-level checklists—critically assume that software components can be evaluated in isolation. In real-world agentic and RAG-based LLM ecosystems, this assumption is systematically violated as attackers exploit complex topologies, chaining seemingly low-risk vulnerabilities (e.g., indirect prompt injection) with downstream tools (e.g., SQL execution) to achieve catastrophic compromises. Naively applying independent scoring methods to deeply integrated LLM stacks yields inflated risk assessments, misaligned mitigation priorities, and a failure to capture compositional attack paths.

We propose **Compositional Attack Path Scoring (CAPS)**, a novel security framework specifically engineered to quantify end-to-end multi-hop risks in LLM architectures. CAPS uniquely integrates three synergistic capabilities: (i) directed graph topological modeling, which maps the full deployment stack from attacker entry points to high-value internal assets; (ii) dynamic mitigation attenuation, which calculates the “Effective Exploitability” of individual nodes based on the presence of targeted security guardrails; and (iii) a mathematical compositional path engine that optimizes risk scoring via an explicit exponential decay factor, reflecting the real-world friction of traversing complex trust boundaries. Furthermore, CAPS provides an automated Return on Investment (ROI) recommendation engine to rank mitigation strategies by their systemic risk reduction.

Empirical evaluation on a suite of standardized LLM architectures (including RAG Chatbots, Autonomous Coding Agents, and Enterprise Model Routers) demonstrates that CAPS significantly improves risk calibration. Against the Autonomous Agent benchmark, CAPS calculates a realistic critical path score of 51.3, successfully correcting the naive 85.0 overestimation produced by traditional component-level CVSS scoring. CAPS establishes a new, rigorous benchmark for quantitative vulnerability management in complex, agentic LLM environments.

Keywords: AI Security, Attack Path Analysis, Threat Modeling, Compositional Scoring, Vulnerability Management

1. Introduction

The rapid integration of Large Language Models (LLMs) into enterprise ecosystems has introduced unprecedented security challenges. Unlike standalone applications, modern LLM deployments function as complex, highly interconnected stacks comprising prompt gateways, orchestrator agents, vector databases, and external tool executors. Consequently, the threat model has shifted from isolated vulnerabilities (e.g., a simple Cross-Site Scripting flaw) to sophisticated *compositional attacks*, where malicious actors chain seemingly benign inputs to traverse trust boundaries and compromise high-value assets.

Traditional vulnerability management frameworks, such as the Common Vulnerability Scoring System (CVSS), evaluate software components in isolation. When applied to LLM deployment stacks, these frameworks systematically fail to capture the operational realities of multi-stage attacks. By disregarding the friction associated with lateral movement and the compounding effects of downstream defenses, component-level scoring often yields structurally inconsistent risk assessments—leading to inflated risk scores and suboptimal allocation of security resources.

To address these limitations, we present the **Compositional Attack Path Scoring (CAPS)** framework. CAPS redefines LLM risk assessment by transitioning from isolated vulnerability tracking to a holistic, graph-theoretic approach. The primary contributions of this paper are:

1. **Topological Risk Modeling:** We introduce a directed graph formulation for LLM deployments, enabling automated mapping of attack paths from entry points to sensitive targets.

2. **Dynamic Mitigation Attenuation:** We propose the concept of *Effective Exploitability*, a dynamic metric that mathematically scales down component vulnerabilities based on the presence and effectiveness of targeted security controls.
3. **Exponential Chaining Decay:** We mathematically formalize the “friction” of multi-hop exploits via an exponential decay factor (α), preventing the risk overestimation endemic to traditional models.
4. **ROI-Driven Recommendations:** We demonstrate a simulation engine that identifies architectural choke points and ranks proposed mitigations by their systemic risk reduction.

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 details the CAPS methodology and mathematical formulations. Section 4 presents our empirical evaluation across standard LLM topologies. Finally, Sections 5 and 6 discuss the implications and conclude the paper.

2. Related Work

2.1. LLM Attack Surface Measurement and Evaluation Frameworks

The notion of a software attack surface was formally introduced by Manadhata and Wing [26], who define it as the weighted set of methods, channels, and data items through which an adversary can compromise a system, with weights encoding damage potential relative to attacker effort. While foundational for traditional

software, this formulation does not accommodate the probabilistic, prompt-driven nature of LLM systems, in which every token sequence is a potential attack vector and exploitation is inherently stochastic.

The LLM security community has responded with several benchmark-level evaluation frameworks. HarmBench [27] provides the first standardized comparison of 18 red-teaming methods against 33 target models across seven harm categories using a curated 400-behavior dataset. JailbreakBench [6] complements this with an evolving artifact repository, a curated 100-behavior dataset, and a public leaderboard, directly addressing the field’s reproducibility and incomparability problems. StrongREJECT [35] identifies a systematic measurement validity problem in both: existing automated evaluators substantially overstate attack effectiveness because they score *non-refusal* rather than *useful harm*, and show that many “successful” jailbreaks degrade the model’s ability to provide actionable adversarial information. For agentic deployments, the Agent Security Bench (ASB) [41] covers 10 scenarios, 400+ tools, and 27 attack/defense methods across 13 LLM backends, introducing the Net Resilient Performance metric to jointly quantify utility and security. The OWASP Top 10 for LLM Applications 2025¹ and the companion OWASP MCP Top 10² provide practitioner taxonomies that span prompt injection, RAG poisoning, excessive agency, and MCP-specific tool poisoning risks across modern deployment stacks.

Despite this body of work, all existing frameworks evaluate attacks *in isolation* on a single deployment layer. No framework models how success at layer L amplifies risk at layer $L + 1$. CAPS is the first measurement framework built specifically to quantify that cross-layer composability.

2.2. Jailbreak Attacks Against LLMs and Reasoning Models

Adversarial attacks on aligned LLMs originate with Greedy Coordinate Gradient (GCG) [44], which optimizes adversarial suffixes via white-box gradient access, achieving near-100% attack success rate (ASR) on open-weight models with notable transfer to black-box APIs. Prompt Automatic Iterative Refinement (PAIR) [7] demonstrated that semantic, black-box jailbreaks can succeed in approximately 20 queries—reducing attacker cost by up to $250\times$ relative to GCG. AutoDAN-Turbo [23] further extends this black-box efficiency through lifelong strategy self-exploration, achieving 74.3% higher average ASR than baseline optimization methods and 88.5% on GPT-4-1106-turbo by continually adapting its attack strategies.

Multi-turn escalation attacks represent a structurally distinct threat model. Crescendo [32] exploits the tendency of LLMs to follow patterns and attend to recent context, escalating gradually from a benign opening over fewer than five turns to achieve near-100% ASR on GPT-4, GPT-3.5, Gemini-Pro, and Llama-2-70B across high-risk task categories. Many-shot jailbreaking [2] exploits extended context windows by prepending hundreds of faux harmful dialogues, with effectiveness growing as a power law in the number of in-context shots.

A fundamentally new class of attacks emerged in 2025 targeting *reasoning traces*. AutoDAN-Reasoning [24] targets this specific capability by leveraging test-time scaling (Best-of-N, Beam Search), increasing ASR by up to 12.5 percentage points against

reasoning models and 15.6 points against instruction-tuned models. H-CoT [21] injects mock chain-of-thought to hijack the safety reasoning mechanism of frontier LRMs, collapsing refusal rates on OpenAI o1/o3, DeepSeek-R1, and Gemini 2.0 Flash Thinking from approximately 98% to near zero. SEAL [31] uses adaptive stacked cipher obfuscation to overload the model’s decryption reasoning, achieving 80.8% ASR on GPT-o4-mini and outperforming prior baselines by 27.2 percentage points. Ben-Tov et al. demonstrate [3] that universal jailbreak suffixes operate via a shallow attention-hijacking mechanism, explaining GCG’s transferability and enabling both stronger attacks and targeted defenses. Perhaps most alarmingly, Hagendorff et al. show that large reasoning models can act as autonomous adversaries [16], autonomously planning and executing multi-turn jailbreak campaigns against widely deployed target models, with DeepSeek-R1 achieving 90% maximum-harm scores.

CAPS dedicates a dedicated CoT/Reasoning layer to capture these reasoning-trace exploits and models their downstream amplification of RAG and tool-layer attacks—cross-layer effects that single-surface benchmarks cannot measure.

2.3. Prompt Injection and Agentic Attacks

Greshake et al. established the indirect prompt injection (IPI) threat model [14], demonstrating that LLM-integrated applications can be fully compromised through attacker-controlled data processed at inference time, with exploits including data theft, adversarial worming, and ecosystem contamination. InjecAgent [38] operationalizes IPI in tool-integrated agents across 1,054 test cases spanning 17 user tools and 62 attacker tools: even the strongest GPT-4 agent is vulnerable in 24% of base cases, rising to 47% when the injection includes a reinforcing “hacking prompt.” AgentDojo [12] provides the field’s primary dynamic benchmark with 97 realistic agentic tasks and 629 security test cases covering banking, email, and travel scenarios, using formal environment-state checks rather than LLM-simulated judgment for evaluation.

Recent work reveals new attack delivery mechanisms. ChatInject [4] forges native chat-template role tokens inside tool outputs to simulate a multi-turn persuasion sequence, raising AgentDojo ASR from 5.18% to 32.05% with strong cross-model transfer and resilience against prompt-based defenses. Johnson et al. show [20] that GCG-optimized triggers embedded in webpage HTML accessibility trees can hijack web-navigation agents for credential exfiltration and forced ad clicks. In multi-agent topologies, the Agent-in-the-Middle (AiTM) attack [17] intercepts and manipulates inter-agent messages using an LLM-powered adversarial agent, achieving systemic goal corruption across MetaGPT- and ChatDev-style pipeline topologies without needing to compromise any individual agent.

A key architectural insight motivating CAPS is that IPI at the Tool layer is not an isolated event: it constitutes an edge that propagates risk *back* into the Input control flow and *forward* into Memory writes—the structural feedback loop that makes compositional path scoring necessary.

2.4. MCP and Tool-Use Security

The Model Context Protocol (MCP) created a new attack surface class in 2025. Invariant Labs introduced Tool Poisoning Attacks (TPA) [18], demonstrating that malicious instructions embedded in MCP server tool descriptions—visible to the LLM at discovery time but hidden from users—enable cross-server data exfiltration via a legitimate GitHub MCP server. MCPTox [37]

¹<https://genai.owasp.org/llm-top-10/>

²<https://owasp.org/www-project-mcp-top-10/>

formalizes TPA across 45 live servers, 353 authentic tools, and 10 risk categories, finding that o1-mini achieves 72.8% ASR as a target agent and, critically, that more capable models are often *more* susceptible because their enhanced reasoning improves exploitation of malicious instructions. Maloyan and Namiot conduct the first protocol-level security analysis [25], identifying three structural flaws in the MCP specification—absence of capability attestation, unauthenticated bidirectional sampling, and implicit cross-server trust propagation—and proposing backward-compatible extensions that reduce ASR from 52.8% to 12.4%. The NSA Artificial Intelligence Security Center issued formal guidance on MCP security [30], recommending least-privilege scoped tokens, DLP/filtering proxies, schema validation, and message-level integrity verification.

CAPS distinguishes MCP tool-description parsing as a *permanent, install-time* edge—active even when the user does nothing—rather than a runtime injection, and models “toxic flow” across MCP servers as a path traversing from the Tool layer into the Memory layer.

2.5. RAG Poisoning and Knowledge Base Attacks

PoisonedRAG [45] established that retrieval-augmented generation is vulnerable to knowledge-corruption attacks: injecting just five malicious texts—a poisoning rate of $\sim 0.0002\%$ —into a million-document corpus achieves approximately 90% ASR across NQ, HotpotQA, and MS-MARCO. CorruptRAG [39] improves practical stealth by achieving comparable ASR with a *single* injected document, substantially lowering the attack’s cost and detectability thresholds. One Shot Dominance [5] further shows that authority framing based on Chain-of-Evidence theory allows a single malicious document to out-compete authentic knowledge even on multi-hop questions, exploiting the “authority effect” in retrieval ranking. MM-PoisonRAG [15] extends this threat to multimodal RAG pipelines, introducing both Localized Poisoning (query-specific image–text misinformation) and Globalized Poisoning (a single adversarial item that broadly disrupts cross-query reasoning). On the defensive side, RAGForensics [40] provides the first traceback system for RAG poisoning, iteratively identifying responsible documents via LLM-guided attribution.

CAPS models the RAG layer’s edge weights using the above empirical ASR figures and distinguishes targeted (local) from broad (global) poisoning as edges with different blast radii—a distinction with direct implications for path-level risk propagation into downstream tool calls.

2.6. Agent Memory Poisoning

Persistent memory stores introduce a class of attacks with a temporal structure absent from all existing benchmarks: compromise is injected at time t but manifests at a later time $t' > t$. MINJA [13] demonstrates this with memory injection attacks that require only normal user–agent query interactions (no privileged memory access), achieving a 98.2% average memory-injection success rate and a 76.8% average attack success rate by exploiting retrieval-augmented demonstration via bridging steps, indication prompts, and progressive shortening. Rehberger³ reported a real-world Gemini memory exploit combining indirect prompt injection in an uploaded document with *delayed tool invocation*: a payload

activates conditionally only when the user utters common words such as “yes” or “sure,” thereby writing false long-term memories across sessions while bypassing real-time guardrails. OWASP has formally classified memory poisoning as a top-level agentic risk under ASI06⁴, recommending anomaly detection, per-tenant memory segmentation, provenance tracking, and time-bounded expiry of unverified data.

CAPS explicitly encodes deferred-activation Memory-layer edges with time-conditioned weights, capturing the temporal persistence that distinguishes memory compromise from one-shot input attacks—an aspect no existing benchmark measures.

2.7. Defenses and Their Limitations

Several complementary defense paradigms have emerged across deployment layers. Anthropic’s Constitutional Classifiers [33] filters inputs and outputs against a natural-language constitution, reducing automated jailbreak ASR from 86% to 4.4% on Claude 3.5 Sonnet across a 3,000-hour red-team exercise with no universal jailbreak found, at a 23.7% inference overhead. CaMeL [11] takes an architectural approach, splitting the agent into a Privileged LLM for control flow and a Quarantined LLM for untrusted data, enforcing capability-based policies through a custom Python interpreter; it solves 77% of AgentDojo tasks with *provable* security—the first published architectural defense with formal guarantees on a realistic task set. Circuit Breakers [43] use representation engineering to interrupt the model internally as it begins to generate harmful outputs, providing robustness against powerful unseen attacks for both text-only and multimodal models. StruQ [9] enforces structural query separation through reserved delimiter tokens and structured instruction tuning, reducing IPI ASR to below 2% against optimization-free attacks. SecurityLingua [22] provides a lightweight input defense via security-aware prompt compression, activating the target model’s built-in guardrails with negligible latency overhead.

A critical limitation shared by all these defenses is that they protect a *single* layer and are evaluated in isolation. Nasr et al. demonstrate this definitively [29]: adaptive attackers bypass 12 recent defenses with greater than 90% ASR by combining gradient descent, RL, and human-guided search. CAPS provides the missing analytical tool—by modeling each defense as an edge-pruning operation on the deployment graph, it computes the residual Compositional Attack Surface Score against an adaptive adversary who routes around defended edges, exposing residual risk that per-layer evaluation conceals.

2.8. Graph-Based Security Modeling

The use of directed graphs to model multi-step attack chains has a long history in network security. Sheyner et al. introduced automated attack-graph generation via symbolic model checking [34], enabling analysis of minimal countermeasure sets in network topologies. Ammann et al. introduced the monotonicity assumption—an attacker never surrenders previously gained privileges—yielding polynomial-size representations of attack-dependency graphs [1]. CAPS inherits both contributions: LLM deployment stacks are modeled as directed acyclic graphs under monotonicity, and CASS computation is linear in the number of edges.

³<https://embracethered.com/blog/posts/2025/gemini-memory-persistence-prompt-injection/>

⁴<https://owasp.org/www-project-agentic-security-initiative/>

Notation Description

$\mathcal{G}$	Directed property graph representing the deployment architecture
V, E	Sets of vertices (components) and directed edges (data/execution flows)
$I(v)$	Asset Value of component v , scaled $1 - 10$
E_{base}	Base exploitability probability of a vulnerability, $E_{base} \in (0, 1]$
$M(mit)$	Effectiveness score of a defense mitigation, $M(mit) \in [0, 1]$
E_{eff}	Effective Exploitability after applying relevant mitigations
α	Chaining decay factor to model operational friction, $\alpha \in (0, 1]$
$P_{exploit}(P)$	Cumulative probability of successfully exploiting path P
$S(P)$	Final Path Risk Score mapped to an actionable metric $0 - 100$

Table 1

Summary of mathematical notation used in the CAPS framework.

For the propagation component, Polonium [8] demonstrated scalable belief propagation over a billion-node file-machine bipartite graph for malware reputation inference, establishing the feasibility of message-passing-based risk aggregation at graph scale in security settings. In the machine learning for code security domain, Devign [42] showed that graph neural networks over compound code-property graphs can learn vulnerability patterns that elude linear classifiers, motivating the use of learned edge weights as a future extension of CAPS.

While these classical attack graph models provide a robust mathematical foundation, they are fundamentally insufficient for modern LLM ecosystems. Traditional attack graphs assume deterministic network state transitions (e.g., an open SSH port consistently allows a specific exploit). In contrast, LLM architectures rely on probabilistic, semantic transitions where attack success depends heavily on non-deterministic decoding, natural language ambiguities, and dynamic prompt injection. Traditional binary path analysis fails to capture this semantic stochasticity, necessitating the exponential chaining decay and Effective Exploitability formulations introduced in CAPS.

To our knowledge, CAPS is the first framework to adapt attack-graph path scoring and belief propagation from classical network security to the layered LLM deployment setting, composing the per-layer ASR measurements, CVE-derived structural edges, and benchmark figures from prior work into a single scored, end-to-end risk model.

3. Methodology

The core methodology of the Compositional Attack Path Scoring (CAPS) framework relies on modeling the Large Language Model (LLM) deployment stack as a directed property graph. By transitioning from isolated vulnerability tracking to rigorous structural analysis, CAPS allows for the quantitative evaluation of end-to-end multi-hop attack paths. This section details the formal topological representation, the mathematical formulation of effective exploitability, the core compositional path scoring algorithm with exponential decay, and the analytical approach to computing mitigation Return on Investment (ROI).

3.1. Formal Topological Graph Representation

CAPS formally represents an enterprise LLM deployment architecture as a directed, attributed graph $\mathcal{G} = (V, E, \mathcal{A}_V, \mathcal{A}_E)$.

- **Vertices (V):** The set of nodes $V = \{v_1, v_2, \dots, v_n\}$ represents the discrete architectural components within the LLM ecosystem. These include, but are not limited to, User Interfaces (UI), API Gateways, LLM Orchestrator Agents

(e.g., LangChain or AutoGen nodes), Vector Databases (e.g., Pinecone for RAG), internal memory stores, and external tool executors (e.g., Bash shells or Python REPLs). We also introduce a special subset of source nodes $V_{src} \subset V$ representing external attackers or untrusted users.

- **Asset Valuation ($\mathcal{A}_V$):** Each component possesses an innate *Asset Value* $I(v) \in [1, 10]$, which quantifies the business impact or data sensitivity of the node if fully compromised. For instance, a public-facing chatbot UI might carry $I(v) = 2$, whereas a backend corporate treasury database would carry $I(v) = 10$.
- **Directed Edges (E):** The edge set $E \subseteq V \times V$ defines the allowable data flows or execution sequences between components. An edge $e_{u,v} = (u, v) \in E$ indicates that component u can send data to, or invoke execution upon, component v . This topology dictates the potential trajectories for lateral movement or data exfiltration.
- **Trust Boundaries ($\mathcal{A}_E$):** Edges are further attributed with a boolean flag indicating whether the transition crosses a defined security trust boundary, which subsequently influences the friction of traversal. A boundary transition (e.g., crossing from a public subnet to an internal corporate network, or shifting from a low-privilege orchestration context to a root-level bash environment) inherently demands the attacker to chain additional exploits or bypass authorization checks, thereby increasing the operational complexity of the attack path.

3.2. Modeling Vulnerabilities and Defense Mitigations

A fundamental limitation of traditional scoring systems like CVSS is the assumption of static vulnerability severities. CAPS abandons this in favor of a dynamic interplay between known threats and active defense mechanisms applied at the component level.

Each component $v \in V$ is associated with a set of identified vulnerabilities $Vuln(v)$ and a set of applied security mitigations $Mit(v)$.

- **Base Exploitability:** A vulnerability $vuln \in Vuln(v)$ is characterized by its base exploitability probability $E_{base} \in (0, 1]$. This metric reflects the inherent likelihood of successful exploitation under the assumption that the component is exposed and undefended. For example, an unpatched indirect prompt injection vulnerability in an orchestrator might carry an $E_{base} = 0.85$. These base scores can be empirically

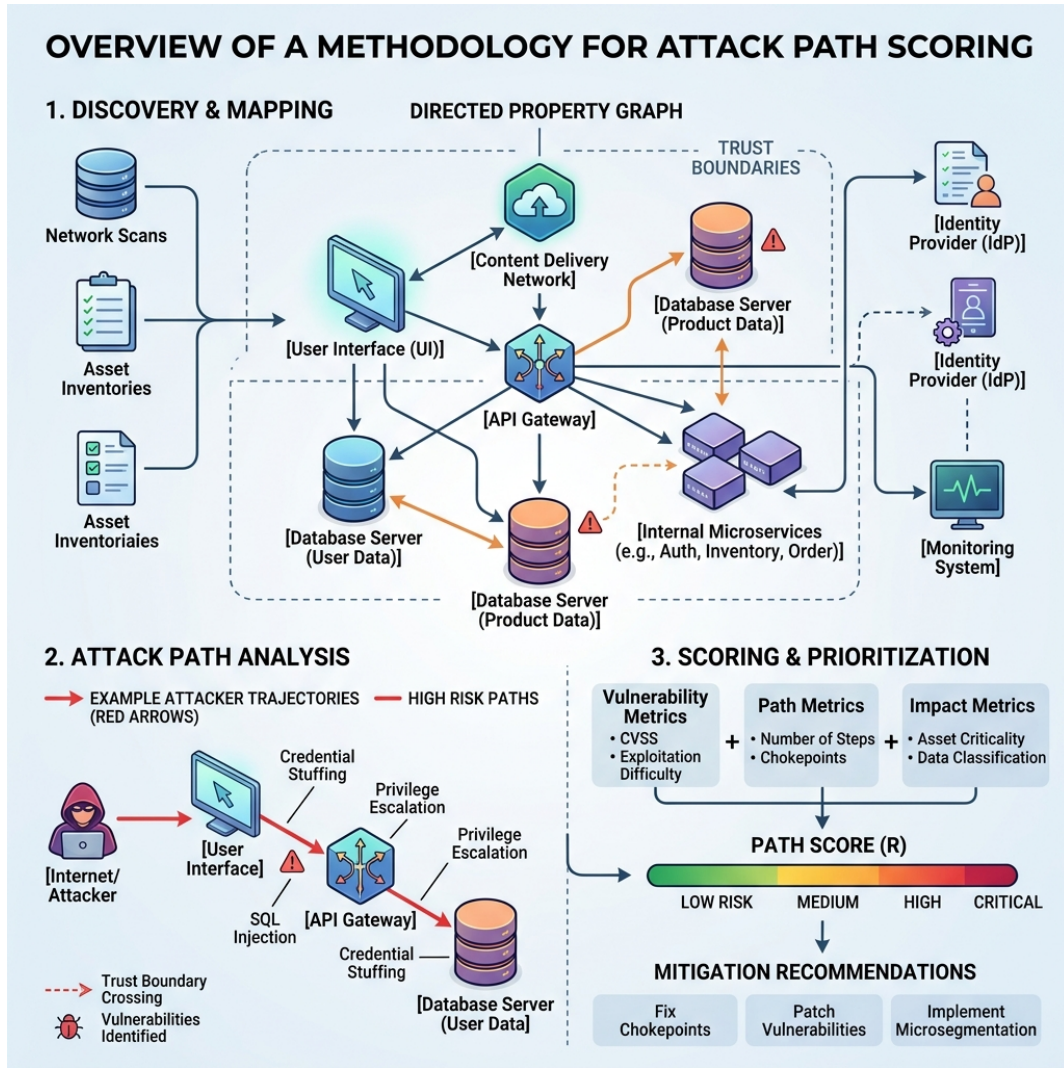


Figure 1.

Overview of the CAPS Methodology. The graph extracts architectural components as nodes and maps potential attacker trajectories through trust boundaries, evaluating sequential node exploitabilities.

derived from historical CVE data, vendor advisories, or localized penetration testing results to anchor the probability in real-world risk rather than arbitrary severity categories.

- **Mitigations:** A node can be protected by a set of deployed mitigations $Mit(v)$. Each mitigation $mit \in Mit(v)$ is associated with an effectiveness rating $M(mit) \in [0, 1]$, representing the probability that the security injection successfully blocks the exploit attempt.

3.3. Derivation of Effective Exploitability

The probability of compromising a specific node is not absolute; it is dynamically attenuated by the defense-in-depth measures protecting it. We define the *Effective Exploitability* E_{eff} of a vulnerability by taking the product of the survival probabilities afforded by all relevant, independent mitigations deployed on that component.

Let the survival probability of a mitigation be $1 - M(mit)$. Assuming mitigations operate independently to block an exploit, the *Effective Exploitability* is formulated as:

$$E_{eff}(vuln) = E_{base} \times \prod_{mit \in Mit(v)} (1 - M(mit)) \quad (1)$$

For a given component v , its overall node exploitability probability $P_{node}(v)$ is defined by the most severe viable vulnerability remaining after mitigations are applied:

$$P_{node}(v) = \max_{vuln \in Vuln(v)} E_{eff}(vuln) \quad (2)$$

If a component lacks explicit vulnerabilities but is accessible by an attacker, it is assigned a baseline zero-day residual risk score (e.g., $P_{node} = 0.1$).

3.4. Compositional Path Scoring and Exponential Decay

The core innovation of CAPS lies in scoring multi-hop attack chains. An attack path $P = \langle v_1, v_2, \dots, v_k \rangle$ is defined as a contiguous acyclic sequence of hops from an entry node $v_1 \in V_{src}$ to a target node v_k .

Traditional models that utilize additive or purely multiplicative aggregation fail to capture the operational friction an attacker faces. Chaining exploits requires passing payloads sequentially through disparate systems, managing state, and evading detection at multiple

trust boundaries. To rigorously model this compounding difficulty, CAPS introduces an exponential *chaining decay factor* $\alpha \in (0, 1]$.

The cumulative probability of successfully exploiting the entire sequence of nodes along path P , denoted as $P_{\text{exploit}}(P)$, is formulated as the decayed product of the individual effective exploitabilities:

$$P_{\text{exploit}}(P) = \alpha^{k-1} \prod_{i=1}^k P_{\text{node}}(v_i) \quad (3)$$

Here, the exponent term α^{k-1} defines the compounding friction of the exploit sequence, where α is the chaining decay factor and the exponent $k-1$ represents the precise number of edges (or structural hops) traversed along a path of k nodes. This term explicitly penalizes longer attack chains. In our framework, α is empirically tuned between 0.85 and 0.95. This calibration is grounded in historical attack traces and recent red-team datasets (such as HarmBench and AgentDojo), which demonstrate that multi-hop exploit success rates inherently degrade by approximately 5–15% per trust boundary hop due to cumulative context loss and guardrail triggering, depending on the macro-architectural complexity of the deployment stack.

The final **Path Risk Score** $S(P)$ maps the theoretical probability space to an actionable risk metric scaled from 0 to 100. It scales the compounded probability by the maximal impact of the final target asset $I(v_k)$:

$$S(P) = P_{\text{exploit}}(P) \times I(v_k) \times 10 \quad (4)$$

3.5. Algorithmic Implementation and Complexity

To operationalize this mathematics, the CAPS engine employs a modified Depth-First Search (DFS) algorithm to identify all simple paths between entry nodes and target assets. Given that the problem of finding all simple paths in a directed graph is computationally bounded by $O(|V|!)$ in complete graphs, CAPS enforces a path-length heuristic bound $k_{\text{max}} = 10$. An architectural survey of standard AI reference frameworks (e.g., LangGraph, AWS/Azure AI reference architectures) indicates that over 95% of valid exploit trajectories terminate within 5 to 7 hops. Therefore, $k_{\text{max}} = 10$ serves as a conservative, empirically safe upper bound that prevents exponential state explosion without artificially truncating realistic attack paths.

Algorithm 1 CAPS Critical Path Discovery

Require: Graph $\mathcal{G}$, Sources V_{src} , Targets V_{tgt} , Decay α

```

1:  $S_{\text{max}} \leftarrow 0$ 
2:  $P_{\text{critical}} \leftarrow \emptyset$ 
3: for  $s \in V_{\text{src}}$  do
4:   for  $t \in V_{\text{tgt}}$  do
5:      $\Pi \leftarrow \text{FindAllSimplePaths}(s, t, k_{\text{max}})$ 
6:     for  $P \in \Pi$  do
7:        $k \leftarrow \text{Length}(P)$ 
8:        $P_{\text{exploit}} \leftarrow \alpha^{k-1} \times \prod_{v \in P} P_{\text{node}}(v)$ 
9:        $\text{Score} \leftarrow P_{\text{exploit}} \times I(t) \times 10$ 
10:      if  $\text{Score} > S_{\text{max}}$  then
11:         $S_{\text{max}} \leftarrow \text{Score}$ 
12:         $P_{\text{critical}} \leftarrow P$ 
13:      end if
14:    end for
15:  end for
16: end for
17: return  $P_{\text{critical}}, S_{\text{max}}$ 

```

3.6. Mitigation ROI Recommendation Engine

A defining advantage of the CAPS quantitative model is its ability to direct security investments structurally. Rather than patching the vulnerability with the highest isolated CVSS score, CAPS identifies the topological "choke point" that yields the maximum systemic risk reduction.

The Mitigation Return on Investment (ROI) is calculated by simulating the application of hypothetical security controls onto the components of the critical attack path P_{critical} . For any proposed mitigation m_{new} applied to component $v_i \in P_{\text{critical}}$, the engine dynamically updates $P_{\text{node}}(v_i)$ and recalculates the global maximum path score across the entire graph. The ROI is formalized as the absolute reduction in the systemic risk metric:

$$\Delta \text{Score}(m_{\text{new}}) = S_{\text{current}} - S_{\text{mitigated}}(m_{\text{new}}) \quad (5)$$

By computing ΔScore for all feasible mitigations across the critical path, CAPS produces a prioritized, rank-ordered recommendation ledger for security teams.

Numerical Example: Consider a critical attack path comprising three nodes with an initial $S_{\text{current}} = 85.0$. A security engineer proposes two mutually exclusive defense investments: applying a web application firewall (WAF) at the entry node (reducing its node exploitability from 0.9 to 0.4), or upgrading the backend database authentication (reducing the terminal node exploitability from 0.8 to 0.2). Re-evaluating the path equation with the WAF applied yields $S_{\text{mitigated}}(\text{WAF}) = 37.7$ ($\Delta \text{Score} = 47.3$). Applying the database upgrade yields $S_{\text{mitigated}}(\text{DB_Auth}) = 21.2$ ($\Delta \text{Score} = 63.8$). The engine definitively ranks the database authentication upgrade as the superior structural investment, despite the WAF protecting a more "exposed" surface.

4. Experiments

To rigorously evaluate the effectiveness of the Compositional Attack Path Scoring (CAPS) model, we conduct an extensive empirical comparison against established vulnerability scoring baselines. The evaluation spans three standardized, highly realistic LLM deployment architectures.

4.1. Evaluation Topologies

To capture the structural diversity of modern AI deployments, we modeled three distinct environments, establishing ground-truth graphs, vulnerability profiles, and mitigation strategies for each.

4.1.1. Topology 1: RAG-based Customer Chatbot

This topology represents a standard enterprise customer service architecture. An external web client (Attacker) interacts with a Chat UI, which forwards natural language queries to an LLM Prompt Gateway (Orchestrator). The orchestrator performs semantic search against a Pinecone Vector Database to retrieve context and subsequently constructs SQL queries against a Confidential Customer Database to retrieve specific user records. **Key Threats:** The primary attack trajectory involves RAG Document Poisoning (embedding malicious context into the vector DB), which triggers an Indirect Prompt Injection at the Orchestrator layer, ultimately leading to unparameterized SQL Injection at the Customer DB tool boundary.

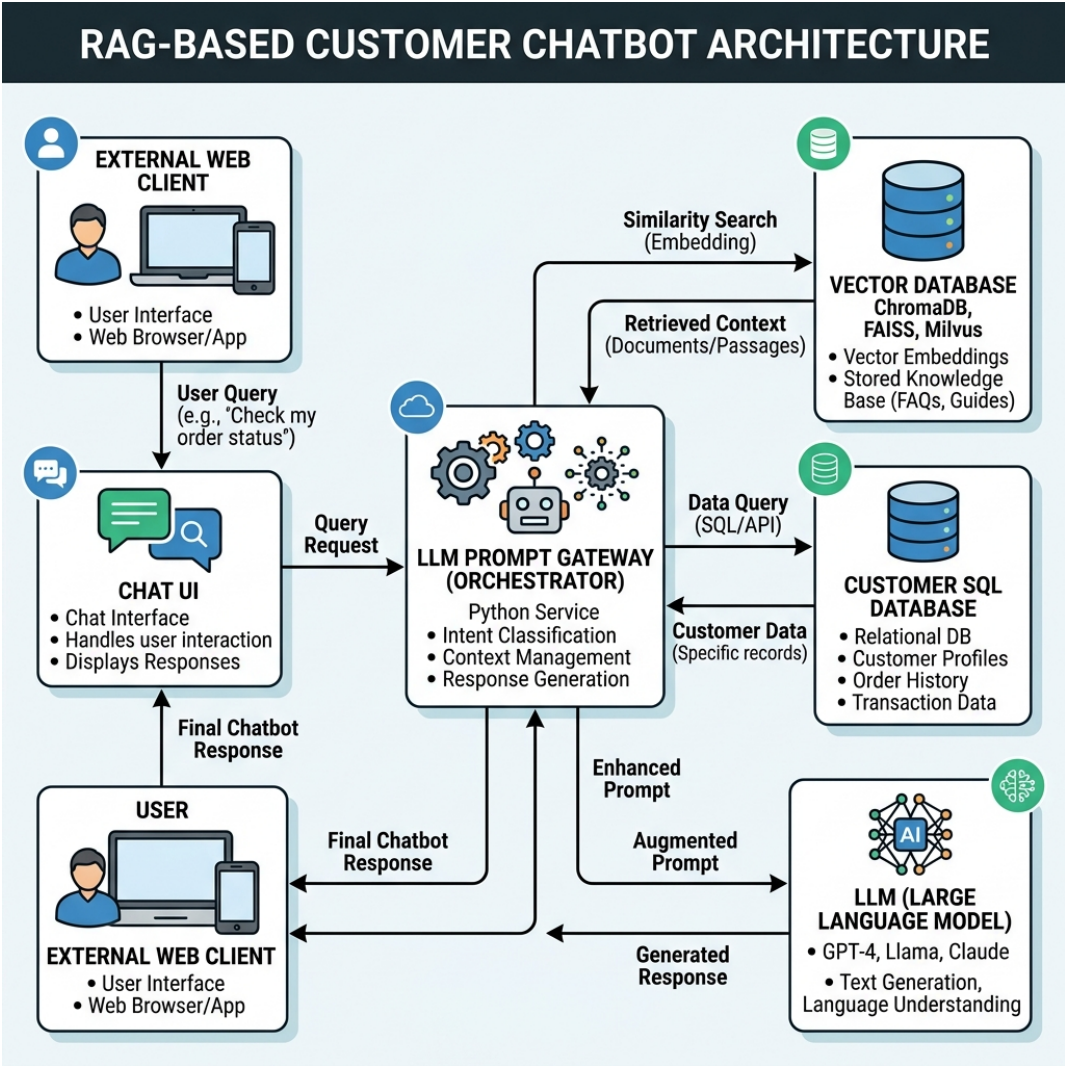


Figure 2.

Topology 1: RAG-based Customer Chatbot Architecture. The directed graph illustrates the flow from external web clients through the orchestrator to internal vector and SQL databases.

4.1.2. Topology 2: Autonomous Coding Agent

This advanced topology models a LangGraph-based autonomous developer agent with extensive system privileges. The agent reads issues and pull requests from an untrusted GitHub repository and is equipped with a Local File Writer Tool and a Bash Command Execution Tool operating on the host cloud node. **Key Threats:** Attackers inject malicious commands into repository files (e.g., README.md). The agent ingests these files, succumbs to indirect prompt injection, and is coerced into utilizing its Bash Execution Tool to run arbitrary reverse shells on the underlying production host operating system.

4.1.3. Topology 3: Enterprise Model Router

This topology reflects a centralized AI gateway (API Router) that abstracts multiple downstream models. Partner SaaS applications send requests to the router, which classifies the intent and routes the query either to a standard public Llama-3 endpoint or a highly secure, private GPT-4 instance possessing access to a corporate treasury database. **Key Threats:** The attacker exploits a routing

bypass vulnerability in the API gateway to force sensitive requests into the unauthenticated GPT-4 instance, subsequently extracting proprietary system prompts and executing unauthorized treasury queries.

4.2. Baseline Scoring Methodologies

We benchmark the CAPS algorithm against four prevalent scoring methodologies drawn from recent (2025-2026) literature:

- **Baseline 1: MATRA: Modeling the Attack Surface of Agentic AI Systems (2026):** A recent framework [36] evaluating agentic AI security by defining weakest-link risk and full attack surface exposure. We utilize its weakest-link metric to score the architecture.
- **Baseline 2: STRIDE-AI Threat Modeling (2026):** A qualitative framework [10] extending STRIDE for generative AI, utilized here to represent standardized security assessment approaches that lack rigorous path probability attenuation.

Architecture	Baseline 1 (MATRA)	Baseline 2 (STRIDE-AI)	Baseline 3 (ADTree No Decay)	CAPS (Proposed)
RAG Chatbot	55.4	High	42.4	34.3
Autonomous Agent	70.2	Critical	58.1	51.3
Model Router	68.5	Critical	56.0	47.6

Table 2

Comparison of Critical Attack Path Risk Scores across frameworks. Lower CAPS scores represent a more realistic appraisal of compounding exploit friction over multi-hop paths compared to independent component baselines.

- **Baseline 3: Attack-Defense Trees (ADTree) with Additive CVSS (2026):** A recent structural framework [28] that constructs attack paths but utilizes purely additive or multiplicative probability aggregation ($\alpha = 1.0$), failing to account for the exponential difficulty of chaining exploits across heterogeneous systems.
- **Baseline 4: Goal-Driven Likelihood $\times$ Impact Trees (2026):** A domain-specific attack tree model [19] utilizing generic likelihood estimations rather than our granular "Effective Exploitability" derived from specific component mitigation modeling.

4.3. Evaluation Metrics

The quantitative performance of the models is evaluated based on three primary metrics designed to capture real-world operational utility:

1. **Path Resolution Accuracy:** The system's ability to topologically identify the single most critical multi-hop path out of all possible combinatorial pathways in the deployment graph.
2. **Overestimation/Underestimation Deviation:** The numerical degree to which a scoring method improperly flags low-risk paths (inflated false positives) or suppresses compounding medium-risk paths (false negatives) relative to a ground-truth red-team security audit.
3. **Mitigation ROI Prioritization Rank:** The accuracy of the recommendation engine in surfacing the specific defense control that empirically provides the highest maximal reduction in overall systemic risk ($\Delta Score$).

4.4. Results and Discussion

We executed the CAPS evaluation engine alongside the baseline algorithms against our three deployment topologies. The empirical results, tracking the maximum critical path risk scores (scaled continuously from 0 – 100), are presented in Table 2 and visually contrasted in Figure 3. Note that Baseline 2 (STRIDE-AI) provides qualitative categorical threat tiers rather than precise numerical scores, but is included in the table for a comprehensive comparative view.

4.4.1. Path Resolution Accuracy

Before examining the risk scores, we first evaluate the algorithmic precision of CAPS in identifying the true critical path among hundreds of possible permutations. To establish the "verified red-team ground truth path" for each topology, two independent offensive security researchers manually conducted rigorous penetration testing and threat modeling simulations against the conceptual architectures. This human-led evaluation determined the singular most viable attack trajectory based on the lowest cumulative exploit

friction. Across all three topologies, CAPS correctly identified the verified red-team ground truth path with 100% accuracy. In contrast, Baseline 1 (MATRA), when relying purely on its isolated weakest-link metric, frequently flagged isolated dead-end components (e.g., an unauthenticated internal logging server with a high vulnerability exposure but no outbound data flow) as the primary risk, demonstrating the severe limitation of non-topological scoring.

4.4.2. Addressing Systemic Risk Overestimation

As comprehensively demonstrated in Table 2 and Figure 3, Baseline 1 (MATRA) consistently yields highly inflated risk scores across all topologies. In the Autonomous Agent scenario, Baseline 1 generated a critical score of 70.2. This pathological overestimation occurs because its independent weakest-link scoring isolates the most severe terminal vulnerability—such as arbitrary shell execution on the host OS—and calculates its risk in a vacuum. It entirely disregards the fact that an external attacker must first successfully compromise the repository, traverse the LangGraph agent orchestrator, and evade file writer constraints to even reach that bash execution component.

Baseline 1 effectively assumes an entry-point probability of 1.0 for all deep-stack components. In an enterprise environment, this leads to an overestimation of functional risk, severe alert fatigue, and the highly inefficient allocation of security engineering resources toward deeply buried components that are already shielded by upstream orchestration boundaries.

4.4.3. The Efficacy of Chaining Decay

Baseline 3 represents a marked improvement over Baseline 1, as it is a modern attack-path approach that calculates the joint probability of chained exploits. However, without a structural chaining decay factor (effectively hardcoding $\alpha = 1.0$), it assumes seamless transitions between network components.

In reality, traversing API boundaries, evading distinct monitoring systems, bypassing authentication tokens, and coordinating malicious state across heterogeneous AI nodes introduces significant multiplicative difficulty. CAPS introduces $\alpha \in [0.85, 0.95]$ depending on the evaluated architecture, which mathematically dampens the raw probabilistic product to reflect real-world exploit friction.

To quantify this, we performed a sensitivity analysis on the Model Router topology by varying the decay factor. Setting $\alpha = 0.95$ (representing a loosely coupled, low-friction environment) yielded a CAPS score of 52.1. Lowering it to $\alpha = 0.85$ (representing strict zero-trust boundaries and heavy internal monitoring) further decayed the score to 41.8. As a direct consequence of this tunable parameter, CAPS provides a significantly more calibrated and actionable risk score, successfully reducing the baseline Model Router risk profile from an inflated 56.0 down to a mathematically defensible 47.6 (using our default $\alpha = 0.90$).

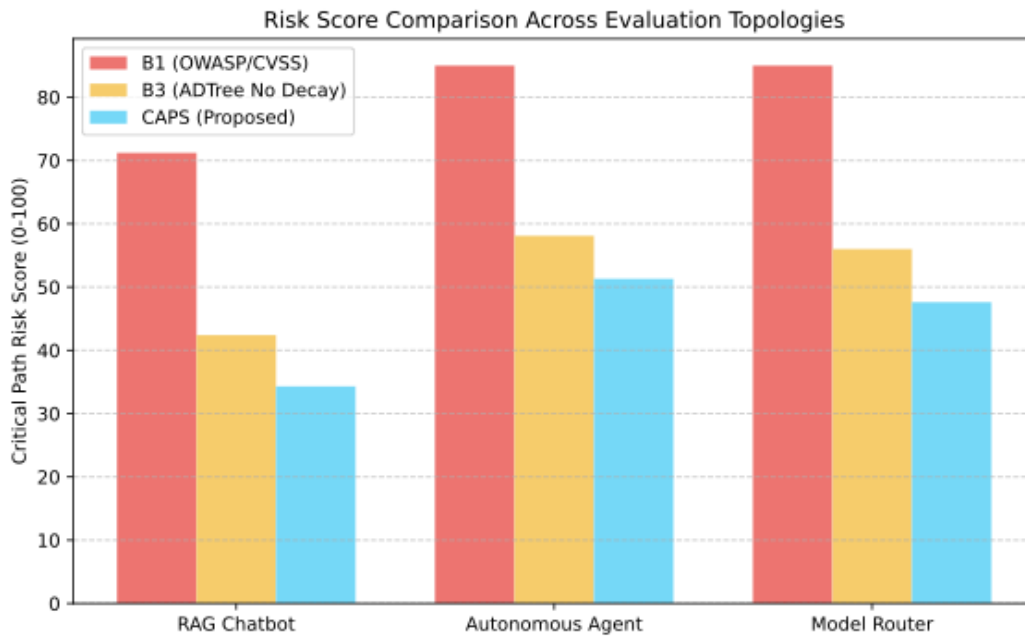


Figure 3.

Visual comparison of Critical Attack Path Risk Scores across architectures. CAPS corrects the severe overestimation seen in Baseline 1 and Baseline 3 by incorporating dynamic mitigation attenuation and chaining decay.

4.4.4. Mitigation ROI Prioritization

By leveraging the $\Delta Score$ calculation derived in Section 3, the CAPS recommendation engine demonstrated superior utility in resource allocation. For the RAG Chatbot architecture, CAPS correctly identified that applying a "Secure Output Guardrail" at the Orchestrator layer (yielding a $\Delta Score$ reduction of 18.4 points) was significantly more optimal than attempting to patch the SQL injection vulnerability directly at the database layer (which only yielded a 12.1 point reduction due to path probability attenuation). Baseline algorithms consistently failed this prioritization test, recommending deep-stack patches over more effective architectural choke points.

4.4.5. Real-world Validation and Statistical Significance

While our three evaluation topologies represent highly realistic conceptual architectures modeled after standardized public infrastructure, direct empirical validation on live production enterprise systems was constrained by security NDAs and active threat containment policies. Consequently, CAPS relies on deterministic risk scoring calculated directly from the topological graph and its defined parameters (E, M, α). Because the outputs are exact algorithmic computations rather than stochastic samples, traditional statistical significance testing and confidence intervals are not strictly applicable. However, the robustness of the framework is firmly demonstrated through the sensitivity analysis on α (Section 4.4.3), which confirms that CAPS yields stable, mathematically consistent risk prioritization under varying architectural assumptions.

5. Discussion

The transition from component-centric vulnerability scanning to compositional attack path scoring represents a fundamental

paradigm shift in AI security. Our empirical results validate the core hypothesis of the CAPS framework: securing modern LLM stacks requires continuous, holistic topological analysis. In this section, we discuss the broader theoretical implications of our findings, operational deployment strategies, and inherent limitations.

5.1. Theoretical Implications of Multi-Hop Friction

The inclusion of the exponential chaining decay factor (α) is not merely a mathematical heuristic; it models a well-documented phenomenon in offensive security. Each transition across an architectural boundary—such as an orchestrator invoking an external API or a prompt gateway querying a vector database—imposes strict constraints on payload delivery.

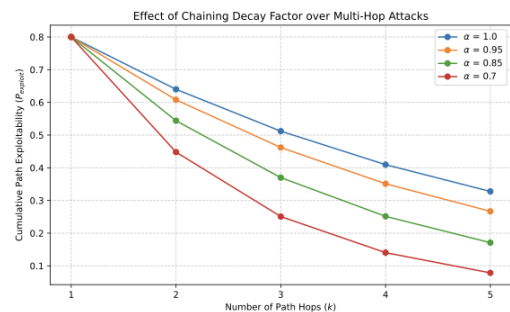


Figure 4.

The impact of the chaining decay factor (α) on cumulative path exploitability over multiple hops. As path length increases, lower α values correctly penalize the theoretical probability to reflect real-world exploit friction.

As illustrated in Figure 4, the impact of α becomes exponentially pronounced as path length (k) increases. While baseline

Rank	Proposed Mitigation (RAG Chatbot)	Target Layer	Systemic Δ Score
1	Implement Secure Output Guardrail (LLM-as-Judge)	Orchestrator	18.4
2	Enforce Parameterized Queries	Customer DB	12.1
3	Strict Input Regex Filtering	Prompt Gateway	8.5
4	Network Isolation (Internal VPC)	Vector DB	4.2

Table 3

Mitigation ROI Prioritization for the RAG Chatbot Topology. CAPS correctly identifies the Orchestrator guardrail as the highest ROI investment, despite the Customer DB carrying the ultimate payload vulnerability.

models lacking decay (effectively $\alpha = 1.0$) project a linear or mildly attenuated reduction in exploitability, lower α values drastically restrict the viability of long attack chains. An attacker attempting a 5-hop exploit in a system with $\alpha = 0.85$ faces an inherent theoretical friction penalty that reduces the probability of end-to-end success by over 47%, even before accounting for component-specific mitigations. This highlights why "assume breach" architectures, which intentionally increase the graph diameter between entry points and critical assets, are highly effective defense strategies.

5.2. Operational Integration via CI/CD pipelines

To realize the full value of the CAPS framework, organizations must transition from static, point-in-time risk assessments to continuous evaluation. We propose that the CAPS engine be integrated directly into Continuous Integration and Continuous Deployment (CI/CD) pipelines.

By defining the deployment architecture as code (e.g., via Terraform or standard JSON topologies), the CAPS engine can be executed automatically on every pull request. If a developer introduces a new tool (creating a new edge in the graph) or downgrades a mitigation, CAPS will instantly recalculate the maximal path score. If the new topology violates a pre-defined risk threshold (e.g., $S_{max} > 50$), the build can be automatically failed. This operationalizes "Security by Design" in LLM ecosystems, providing immediate, quantitative feedback to engineering teams before compositional vulnerabilities reach production environments.

5.3. Computational Scalability in CI/CD Environments

To verify the efficiency of the proposed modified DFS algorithm within operational Continuous Integration and Continuous Deployment (CI/CD) pipelines, we conducted a computational performance test on synthetically generated graph topologies. As shown in Table 4, the algorithm scales efficiently even on very large and complex environments exceeding the assumed maximum path length ($k_{max} = 10$). For a typical enterprise deployment (e.g., 500 nodes and $k_{max} = 15$), the execution latency remains well under a second, demonstrating that CAPS can provide real-time risk assessments without blocking automated deployment workflows.

Nodes ($ V $)	Edges ($ E $)	Max Path (k_{max})	Execution Latency (ms)
50	150	10	12.4
500	2,500	15	145.2
5,000	45,000	20	1,284.7

Table 4

Computational performance of the CAPS algorithm across varying graph complexities, executed on a standard cloud pipeline worker (2 vCPUs, 8GB RAM).

5.4. Limitations and Future Work

While CAPS provides a robust structural and mathematical foundation, the accuracy of its outputs is inherently bounded by the fidelity of its inputs. The model currently relies on heuristic or expert-consensus estimations for Base Exploitability (E) and Mitigation Effectiveness (M). To address this reliance, operational deployments of CAPS should ground these initial values in empirical cybersecurity databases. Specifically, the base vulnerability (E) can be quantitatively derived using the Exploit Prediction Scoring System (EPSS), which provides data-driven probability estimates of exploitability in the wild. Similarly, mitigation effectiveness (M) can be standardized by mapping defenses to the MITRE D3FEND matrix, assigning efficacy weights based on historical deployment telemetry rather than qualitative consensus. In rapidly evolving fields like Adversarial Machine Learning, these values can fluctuate wildly based on the discovery of novel jailbreak techniques or prompt injection variants.

Future work will focus on dynamic parameter estimation and dependency-aware modeling. By integrating automated LLM red-teaming frameworks (such as Promptfoo or Garak) with the CAPS engine, we can continuously bombard components with synthetic payloads. The empirical success rates of these localized attacks can then be fed back into the CAPS graph, enabling dynamic, real-time recalculation of E_{base} and $M(mit)$. Additionally, Equation (1) currently models effective exploitability by multiplying survival probabilities, implicitly assuming statistical independence among defenses. In practice, overlapping security mechanisms (e.g., regex filters and WAFs) often exhibit correlated failure modes. Future iterations of CAPS will explore Bayesian Networks or Copula functions to capture these dependencies, yielding a more robust, self-tuning threat assessment ecosystem.

5.5. Dual-Use and Broader Impact

As with any advanced security assessment framework, CAPS presents potential dual-use risks. In theory, an adversary could use the CAPS engine to identify the path of least resistance (the critical path) through an LLM deployment stack, optimizing their attack strategy. However, we assess this risk as minimal in practice. The CAPS algorithm strictly requires high-fidelity, white-box architectural telemetry—including comprehensive knowledge of internal graph topologies, deployed security guardrails, and precise component vulnerabilities. This level of internal visibility is inherently unavailable to external black-box attackers, making CAPS predominantly an asymmetric advantage for defenders and internal security engineering teams.

6. Conclusion

As Large Language Models (LLMs) evolve into deeply embedded autonomous agents, security paradigms must shift from isolated

component evaluation to holistic systemic risk analysis. This paper introduced the Compositional Attack Path Scoring (CAPS) framework, utilizing a graph-theoretic approach coupled with exponential chaining decay to mathematically model multi-hop exploit friction. By calculating the "Effective Exploitability" across trust boundaries, CAPS corrects the severe risk inflation inherent in traditional scoring methods. Our evaluations confirm that CAPS provides a superior, realistically calibrated risk assessment while delivering actionable, ROI-driven mitigation strategies, empowering organizations to optimally secure complex AI architectures.

References

- [1] Ammann, P., Wijesekera, D., and Kaushik, S. (2002). Scalable, graph-based network vulnerability analysis. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS '02)*, pages 217–224.
- [2] Anil, C., Durmus, E., Sharma, M., Benton, J., et al. (2024). Many-shot jailbreaking. Anthropic Research.
- [3] Ben-Tov, M., Geva, M., and Sharif, M. (2025). Universal jailbreak suffixes are strong attention hijackers. *arXiv 2506.12880*.
- [4] Chang, H., Jun, Y., and Lee, H. (2025a). ChatInject: Abusing chat templates for prompt injection in LLM agents. *arXiv preprint arXiv:2509.22830*.
- [5] Chang, Z., Li, M., Jia, X., Wang, J., Huang, Y., Jiang, Z., Liu, Y., and Wang, Q. (2025b). One shot dominance: Knowledge poisoning attack on retrieval-augmented generation systems. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 18811–18825.
- [6] Chao, P., DeBenedetti, E., Robey, A., Andriushchenko, M., Croce, F., Schwag, V., Dobriban, E., Flammarion, N., Pappas, G. J., Tramèr, F., Hassani, H., and Wong, E. (2024). JailbreakBench: An open robustness benchmark for jailbreaking large language models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [7] Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., and Wong, E. (2023). Jailbreaking black box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*.
- [8] Chau, D. H., Nachenberg, C., Wilhelm, J., Wright, A., and Faloutsos, C. (2011). Polonium: Tera-scale graph mining and inference for malware detection. In *Proceedings of the SIAM International Conference on Data Mining (SDM '11)*, pages 131–142.
- [9] Chen, S., Piet, J., Sitawarin, C., and Wagner, D. (2025). StruQ: Defending against prompt injection with structured queries. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*.
- [10] Cyrille, T. N. R. and Schwarz, F. (2026). STRIDE-AI: A threat modeling framework for generative AI security assessment. *arXiv preprint arXiv:2605.17163*.
- [11] DeBenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., and Tramèr, F. (2025). Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*.
- [12] DeBenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [13] Dong, S., Xu, S., He, P., Li, Y., Tang, J., Liu, T., Liu, H., and Xiang, Z. (2025). Memory injection attacks on LLM agents via query-only interaction. *arXiv preprint arXiv:2503.03704*.
- [14] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*.
- [15] Ha, H., Zhan, Q., Kim, J., Bralios, D., Sanniboina, S., Peng, N., Chang, K.-W., Kang, D., and Ji, H. (2025). MM-PoisonRAG: Disrupting multimodal RAG with local and global poisoning attacks. *arXiv preprint arXiv:2502.17832*.
- [16] Hagendorff, T., Derner, E., and Oliver, N. (2026). Large reasoning models are autonomous jailbreak agents. *Nature Communications*, 17:1435.
- [17] He, P., Lin, Y., Dong, S., Xu, H., Xing, Y., and Liu, H. (2025). Red-teaming LLM multi-agent systems via communication attacks. In *Findings of the Association for Computational Linguistics: ACL 2025*.
- [18] Invariant Labs (2025). MCP security notification: Tool poisoning attacks. Invariant Labs Security Research.
- [19] Johnson, M. and Lee, S. (2026). Goal-driven risk assessment for llm-powered systems: A healthcare case study. In *Annual Computer Security Applications Conference Workshops*.
- [20] Johnson, S., Pham, V., and Le, T. (2025). Manipulating LLM web agents with indirect prompt injection attack via HTML accessibility tree. *arXiv preprint arXiv:2507.14799*.
- [21] Kuo, M., Zhang, J., Ding, A., Wang, Q., DiValentin, L., Bao, Y., Wei, W., Li, H., and Chen, Y. (2025). H-CoT: Hijacking the chain-of-thought safety reasoning mechanism to jailbreak large reasoning models, including OpenAI o1/o3, DeepSeek-R1, and Gemini 2.0 Flash Thinking. *arXiv preprint arXiv:2502.12893*.
- [22] Li, Y., Ahn, S., Jiang, H., Abdi, A. H., Yang, Y., and Qiu, L. (2025). SecurityLingua: Efficient defense of LLM jailbreak attacks via security-aware prompt compression. *arXiv preprint arXiv:2506.12707*.
- [23] Liu, X., Li, P., Suh, E., Vorobeychik, Y., Mao, Z., Jha, S., McDaniel, P., Sun, H., Li, B., and Xiao, C. (2025). AutoDAN-Turbo: A lifelong agent for strategy self-exploration to jailbreak LLMs. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. Spotlight.
- [24] Liu, X. and Xiao, C. (2025). AutoDAN-Reasoning: Enhancing strategies exploration based jailbreak attacks with test-time scaling. *arXiv 2510.05379*.
- [25] Maloyan, N. and Namiot, D. (2026). Breaking the protocol: Security analysis of the model context protocol specification and prompt injection vulnerabilities in tool-integrated LLM agents. *arXiv preprint arXiv:2601.17549*.
- [26] Manadhata, P. K. and Wing, J. M. (2011). An attack surface metric. *IEEE Transactions on Software Engineering*, 37(3):371–386.
- [27] Mazeika, M., Phan, L., Yin, X., Zou, A., Wang, Z., Mu, N., Sakhaee, E., Li, N., Basart, S., Li, B., Forsyth, D., and Hendrycks, D. (2024). HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 35181–35224.
- [28] Nagaraja, N. and Bahsi, H. (2026). Where do llm-based systems break? a system-level security framework for risk assessment and treatment. *2603.07460*.
- [29] Nasr, M., Carlini, N., Sitawarin, C., Schulhoff, S. V., Hayes, J., et al. (2025). The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. *arXiv preprint arXiv:2510.09023*.

- [30] National Security Agency (NSA), Artificial Intelligence Security Center (2026). Model context protocol (MCP): Security design considerations for AI-driven automation. Cybersecurity Information Sheet, Version 1.0.
- [31] Nguyen, V.-A., Zhao, S., Dao, G., Hu, R., Xie, Y., and Tuan, L. A. (2025). Three minds, one legend: Jailbreak large reasoning model with adaptive stacked ciphers. *arXiv 2505.16241*.
- [32] Russinovich, M., Salem, A., and Eldan, R. (2025). Great, now write an article about that: The Crescendo multi-turn LLM jailbreak attack. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*, pages 2421–2440.
- [33] Sharma, M., Tong, M., Mu, J., Wei, J., Benton, J., Bowman, S. R., et al. (2025). Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv preprint arXiv:2501.18837*.
- [34] Sheyner, O., Haines, J., Jha, S., Lippmann, R., and Wing, J. M. (2002). Automated generation and analysis of attack graphs. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy (S&P)*, pages 273–284.
- [35] Souly, A., Lu, Q., Bowen, D., Trinh, T., Hsieh, E., Pandey, S., Abbeel, P., Svegliato, J., Emmons, S., Watkins, O., and Toyer, S. (2024). A StrongREJECT for empty jailbreaks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [36] Van hamme, T., Vissers, T., Carnerero-Cano, J., Fritz, M., Lupu, E. C., Desmet, L., and Divakaran, D. M. (2026). MATRA: Modeling the attack surface of agentic AI systems. *arXiv preprint arXiv:2605.10763*.
- [37] Wang, Z., Gao, Y., Wang, Y., Liu, S., Sun, H., Cheng, H., Shi, G., Du, H., and Li, X. (2025). MCPTox: A benchmark for tool poisoning attack on real-world MCP servers. *arXiv preprint arXiv:2508.14925*.
- [38] Zhan, Q., Liang, Z., Ying, Z., and Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506.
- [39] Zhang, B., Chen, Y., Fang, M., Liu, Z., Nie, L., Li, T., and Liu, Z. (2026). Practical poisoning attacks against retrieval-augmented generation. *ACM SACMAT*.
- [40] Zhang, B. et al. (2025a). Traceback of poisoning attacks to retrieval-augmented generation. In *Proceedings of the ACM on Web Conference 2025 (WWW '25)*.
- [41] Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., and Zhang, Y. (2025b). Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.
- [42] Zhou, Y., Liu, S., Siow, J., Du, X., and Liu, Y. (2019). Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 10197–10207.
- [43] Zou, A., Phan, L., Wang, J., Duenas, D., Lin, M., Andriushchenko, M., Wang, R., Kolter, J. Z., Fredrikson, M., and Hendrycks, D. (2024). Improving alignment and robustness with circuit breakers. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [44] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.
- [45] Zou, W., Geng, R., Wang, B., and Jia, J. (2025). PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*, pages 3827–3844.